

Automatic Chart Generation in Rhythm Games Using Deep Learning: A Case Study on "Taiko no Tatsujin"

Ho Ethan, Chiang Jimmy
NTUT

Abstract

This study presents Beat Understanding (BU), a deep learning-based framework for automatic chart generation in the rhythm game Taiko no Tatsujin. We propose a comprehensive preprocessing pipeline that incorporates annotation normalization and sequence length alignment to enhance training stability and output quality. To address the challenges of label sparsity and inconsistent sequence lengths, a spectrogram-label alignment mechanism with zero-padding is introduced to ensure input uniformity. Additionally, a novel audio segmentation strategy combined with a temporal error correction algorithm is applied, significantly improving model prediction accuracy.

Experimental results demonstrate that BU achieves strong performance in both hit-type classification and human-like rhythm pattern generation, reaching an average OCHuman score of 0.8426. These findings validate the effectiveness of the proposed slicing and alignment strategies in supporting structural modeling. Overall, our work provides empirical evidence and methodological guidance for advancing automatic chart generation in rhythm-based games.

I. INTRODUCTION

With the rapid advancement of digital music and interactive entertainment, enhancing music content understanding and generation through artificial intelligence (AI) has become a significant interdisciplinary research goal. Driven by the progress in AI and deep learning, effective application of these technologies in Music Information Retrieval (MIR) [1] has emerged as an important research area. Since the late 2010s, deep learning techniques have rapidly entered the MIR field, broadening its scope from traditional melody recognition and genre classification to applications such as audio tagging, automatic accompaniment, and interactive gaming, further advancing technologies like melody tracking, rhythm perception, audio source separation, and audio generation [2], [3]. Among these, Automatic Chart Generation [4], which simultaneously handles audio feature interpretation, rhythm structure analysis, and human behavior simulation, is regarded as a key challenge integrating MIR, deep learning, and human-computer interaction.

Rhythm games offer a unique and representative research platform among various application scenarios. Particularly, games such as *Taiko no Tatsujin* [5], which feature clear rhythmic structures and well-defined annotation formats, provide an ideal experimental setting for automatic chart generation research. Each chart in *Taiko no Tatsujin* corresponds

explicitly to audio segments, rhythmic structures, and hit types, offering quantifiable and highly valuable training data for data-driven model learning. Additionally, community-driven content creation and open competitions yield extensive high-quality chart data, facilitating the development of representative datasets and evaluation benchmarks. Initial studies primarily transformed audio signals into spectrograms, extracting local features using Convolutional Neural Networks (CNN) [6], and subsequently modeling sequential information through recurrent neural networks such as Long Short-Term Memory (LSTM) [7].

Nevertheless, automatic chart generation still faces multiple technical challenges. First, hit annotations within the charts are highly sparse, resulting in significant label imbalance during model training. Second, early research often treated hit points as independent prediction events, neglecting global rhythmic context such as BPM (Beats Per Minute), measures, and musical phrases, thus reducing the musicality and structural coherence of generated charts. Third, although architectures combining CNN and LSTM have been widely employed for rhythm modeling—effectively processing audio data and rhythmic features—current research focuses primarily on extracting audio features and translating them into high-quality, human-like game charts. Models such as Dance Dance Convolution (DDC) [8], proposed by Donahue et al., represent prominent examples of this approach, demonstrating successful application to games like *Dance Dance Revolution*, laying the foundation for subsequent research [9].

In the context of *Taiko no Tatsujin*, the TaikoNation system [10] first introduced the concept of human-like patterning into rhythm game chart generation tasks, employing a CNN+LSTM framework to mimic human arrangers' control of note density and rhythmic fluidity. However, its labeling method relies solely on absolute timing, excluding critical rhythmic information such as BPM, measures, and beats, limiting the structural and rhythmic alignment of generated charts and constraining its capability to capture long-range rhythmic dependencies, especially in music with multiple segments or varied rhythms. To address label sparsity, the AGoT system [11] introduced fuzzy labels to replace binary hit annotations with continuous values, reducing the model's overreliance on non-hit samples during training. This approach utilized ROC-AUC as an evaluation metric, effectively improving training stability. Nonetheless, despite addressing label sparsity, AGoT still struggles to capture long-range rhythmic relationships and rhythm variations effectively.

Although representative models such as TaikoNation and AGoT exist, these models still exhibit limitations in address-

ing label imbalance, neglecting musical rhythmic contexts (e.g., BPM), and capturing long-range dependencies. To overcome these limitations, recent research has introduced Transformer architectures, which utilize self-attention mechanisms that excel in sequence modeling tasks, demonstrating substantial advantages in language and audio tasks, particularly in long-sequence learning and global attention control [12], [13]. Recent studies exploring combinations of CNN and Transformer architectures in image captioning, rhythm recognition, and audio understanding tasks have shown superior structural modeling capabilities compared to LSTM [14]–[16].

Given the emergence of CNN+Transformer architectures, their self-attention mechanisms hold promise in addressing LSTM limitations regarding long-range rhythm modeling. However, systematic comparative and empirical analysis in *Taiko no Tatsujin* chart generation tasks remains insufficient. Transformer architectures are sensitive to dataset size and label density, particularly prone to issues such as attention drift, contextual imbalance, and overfitting when dealing with limited data [17], [18]. Zhang and Liu indicated that although Transformer architectures perform well on large datasets, their effectiveness significantly diminishes with small datasets [19]. Wu et al. similarly observed performance improvements in rhythm perception tasks through combining convolutional layers with Transformer architectures [20].

Addressing these gaps, this study aims to develop a comprehensive automatic chart generation system for *Taiko no Tatsujin*, conducting an in-depth comparative analysis of CNN+Transformer architectures. We introduce preprocessing techniques incorporating BPM alignment, rhythm discretization, and sequence padding, alongside standardized audio spectrogram conversion and annotation formats, eliminating inconsistencies in raw data. Through standardized training and post-processing, this study systematically evaluates architectural performance differences and output quality. Particular attention is paid to the practical behavior of Transformer architectures with small datasets, including training stability, rhythmic accuracy, and structural coherence. Employing quantitative indicators DCRand and OCHuman [10], we comprehensively compare rhythmic structure, similarity to human-generated charts, and rhythmic tolerance.

The contributions of this study include:

- A comprehensive automatic chart generation pipeline for *Taiko no Tatsujin*, integrating data cleaning, rhythm annotation, and spectrogram conversion.
- In-depth analysis of CNN+Transformer architectures under conditions of small datasets and sparse annotations.
- Strategies to mitigate Transformer issues in sparse annotation scenarios, enhancing generation stability and rhythm perception.
- Achieving outstanding OCHuman performance (average score of 0.8426), indicating high similarity to human-generated charts.

Through this methodology, we aim to establish clear benchmarks and practical pathways for deep learning applications in rhythm games, laying robust foundations for automated chart generation and game AI development [21].

III. METHOD

A. System Architecture

The proposed Beat Understanding (BU) automatic chart generation system encompasses several core stages, including dataset collection, synchronized preprocessing of audio and chart files, label creation, audio spectrogram conversion, data padding and compression, and deep learning model training and inference. This pipeline ensures standardized data input into the model, addressing format, length, and alignment discrepancies between original music and chart data, thereby enhancing model training efficiency and prediction accuracy. The overall system architecture is illustrated in Fig. 1.

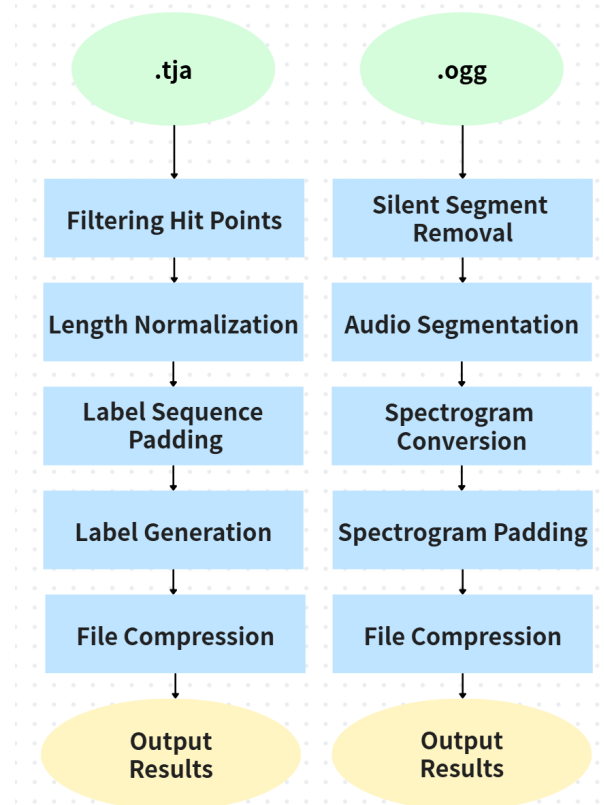


Fig. 1: Overview of the BU system pipeline.

B. Constraints

The BU system is applicable under specific conditions necessary for correct data processing and consistent chart generation. Complete offset and BPM information must be available for accurate alignment of audio and chart data, as missing information impairs synchronization and subsequent processing. Moreover, BU is specifically designed for rhythm games like *Taiko no Tatsujin*, which use a fixed number of discrete hit points per measure (e.g., 16 points). If any measure within a chart cannot be uniformly scaled to 16 equidistant points, the chart is excluded from training data. Additionally, only charts with constant BPM (no BPM changes) are utilized, ensuring data consistency. Charts containing BPM change instructions (e.g., BPMCHANGE) are automatically excluded from the training dataset to avoid alignment errors. These conditions must be strictly met; otherwise, data quality and model training will be compromised.

C. System Architecture

1) *Dataset*: All training data for the BU system are sourced from official *Taiko no Tatsujin* charts, including both arcade and Nintendo Switch versions [22]. We selected charts with difficulty levels ranging from 6 to 7 (hard) for model training. Compared to easy or normal levels, the hard level retains comprehensive rhythmic information beneficial for learning rhythmic structures, while avoiding the BPM variability commonly found in expert-level charts, ensuring data consistency and stability. Through rigorous preprocessing and filtering, we constructed a clearly structured, uniformly formatted, high-quality dataset, enabling effective learning under stable conditions and enhancing the model’s rhythmic prediction and generation stability.

2) *Filtering Hit Points*: Initially, .tja chart files are parsed to obtain BPM, offset, and hit points per measure. Subsequently, each measure’s data is filtered by converting all hit points other than 1 (drum center) or 2 (drum rim) to 0 (no hit), ensuring the model processes only valid hit events, thus reducing noise and enhancing training quality. Before labeling, the program also inspects and removes extraneous measures consisting entirely of zeros at the beginning and end of the chart, preventing these blank segments from affecting label quality and subsequent training. After these steps, only measures containing actual hit information are retained for subsequent processing and training.

3) *Length Normalization*: To eliminate inconsistencies in the number of hits per measure within original charts, each measure’s hit data is normalized into a fixed length of 16 time segments through interpolation (upsampling) or grouped sampling (downsampling). For instance, if a measure originally contains only 4 hit points, it is proportionally replicated and rearranged into 16 segments. Conversely, measures containing more than 16 hit points are downsampled by selecting the maximum value within each group. This step ensures uniformity of measure data format, facilitating efficient batch training for deep learning models. During model inference, reverse scaling restores measures to their original lengths, addressing data length variability and enhancing the stability of the chart generation system.

4) *Label Generation*: Each time segment’s rhythm is analyzed and labeled accordingly. To facilitate multi-class classification training, this study employs one-hot vectors to represent the hit status at each time point. A time segment with a hit value of 0 (no hit) is labeled [0,1,0,0], a hit value of 1 (drum center hit) is labeled [0,0,1,0], and a hit value of 2 (drum rim hit) is labeled [0,0,0,1]. This labeling approach enables the model to distinctly identify each type of hit event, improving accuracy during both training and inference.

5) *Label Sequence Padding*: After labeling hit rhythms, the system calculates the maximum number of hit points across all training songs and uses this value for sequence padding. If a song’s label data does not reach this padding length, padding labels ([1,0,0,0]) are appended to match the required length, ensuring uniformity across all training inputs. This approach prevents variability in hit point counts from negatively affecting model training, thereby enhancing overall training stability and ensuring consistency in the generated charts.

6) *File Compression*: To compress image files or matrices, as shown in Fig. 2, tf.Data’s tf.Features and tf.Example

packages are utilized. Before compression, spectrogram and label matrices are first converted into textual format because tf.Features and tf.Example cannot directly store matrix data. Since the text format converts the original image matrix into a one-dimensional array, the original dimensions of the image matrices must be recorded to accurately reconstruct the three-dimensional matrices upon decompression. Subsequently, tf.Features and tf.Example are applied to compress these textual representations, and the resulting spectrogram text data, label matrices, and their original dimensions are stored as TFRecords files.

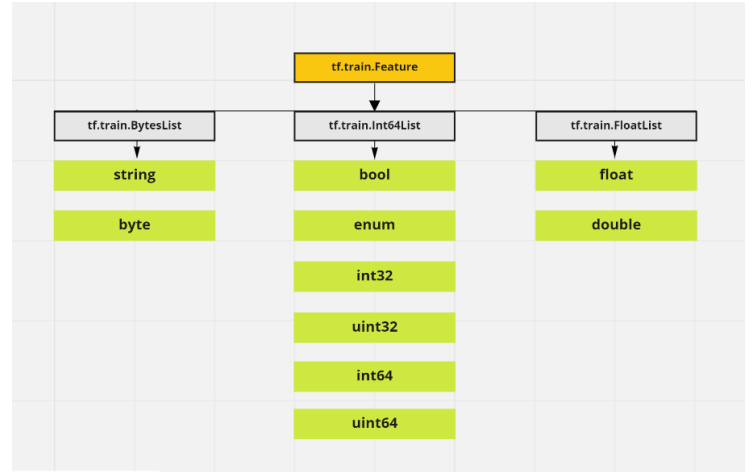


Fig. 2: TFRecord compression pipeline overview.

7) *Silent Segment Removal*: Utilizing the offset obtained through hit point filtering and calculating the total valid chart duration, this step identifies the actual time range of hit events within each song. This allows removal of silent or hitless segments from the beginning and end of audio files during segmentation. Consequently, subsequent data processing and model training focus exclusively on audio content containing chart data, enhancing data quality and training efficiency.

8) *Audio Segmentation*: Songs are segmented into pieces corresponding to the number of valid hit points identified in labeling. By leveraging hit point count and BPM information derived from parsing the chart files, the precise duration for each audio segment is calculated. Due to potential rounding errors in computational precision, cumulative errors may cause misalignment between audio and labels, especially toward the end of songs. To address this, we implemented an error correction method (shown in Fig. 3), where each measure’s duration is rounded to three decimal places. The sum of actual segment lengths is compared to the cumulative ideal length, and discrepancies exceeding 0.1% trigger adjustments in segment length, ensuring continuous alignment. This correction method effectively minimizes cumulative errors, maintaining synchronization between audio segments and labels, and ensures equal numbers of audio and label segments.

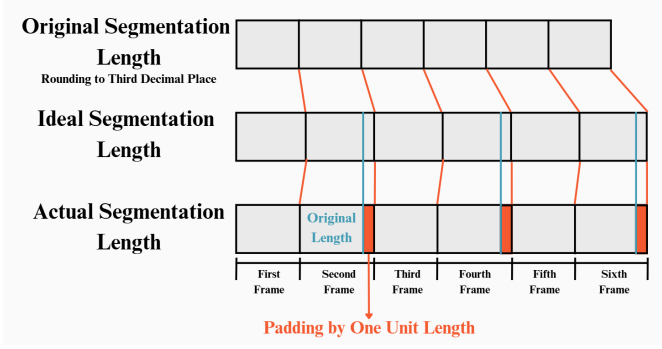


Fig. 3: Segmentation length calculation.

9) *Spectrogram Conversion*: The segmented audio clips are first loaded and converted into numerical arrays. The stereo channels of the audio data are averaged to produce mono-channel audio signals, facilitating subsequent spectrogram conversion. Next, the audio data are normalized to the range $[-1, 1]$, aligning with the interval used by the Fourier transform for further processing. Short-Time Fourier Transform (STFT) is then applied to generate spectrograms, which are converted to a logarithmic scale (dB) to enhance visibility of lower-intensity signals. Each spectrogram is resized proportionally to dimensions of 200×600 pixels, ensuring consistency in the size of training matrices, as illustrated in Fig. 4. Standardizing image dimensions across the dataset facilitates uniformity and simplifies data handling for model input.

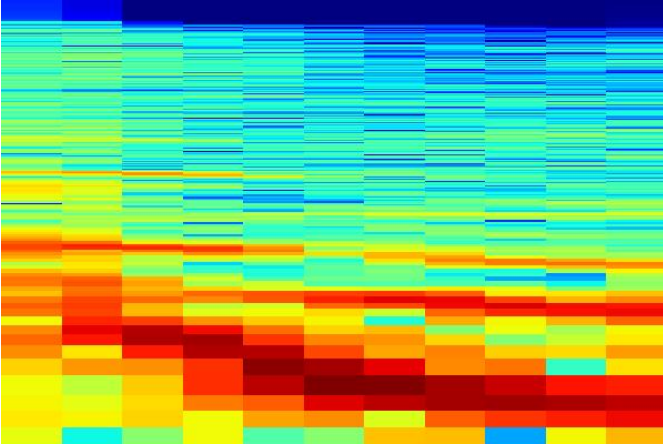


Fig. 4: Example of a spectrogram.

10) *Spectrogram Padding*: As described in *Label Length Padding*, spectrogram data also requires padding, as shown in Fig. 5. When the spectrograms of a particular song are fewer than the standard length, white spectrogram images are appended to the end until their total length matches that of the padded labels. This approach ensures uniformity of dimensions and formatting across all training data, enhancing model training stability and performance. Furthermore, padding with fully white spectrograms explicitly differentiates genuine audio data from padding regions, preventing interference from non-actual data during the learning process.

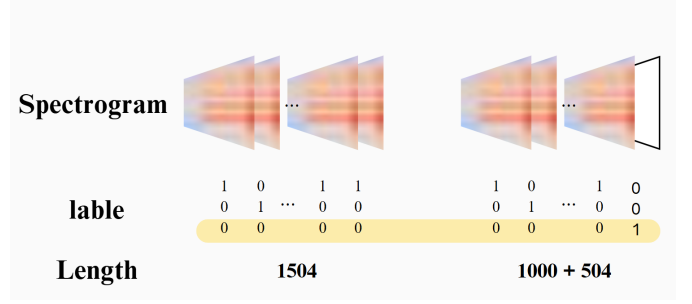


Fig. 5: Spectrogram length padding.

11) *File Compression*: To reduce computational load, spectrograms are initially resized to smaller dimensions (e.g., 60×40 pixels), and pixel values are normalized between 0 and 1. Subsequently, a CNN+Transformer deep learning model architecture is employed for hit-point prediction, as depicted in Fig. 6.

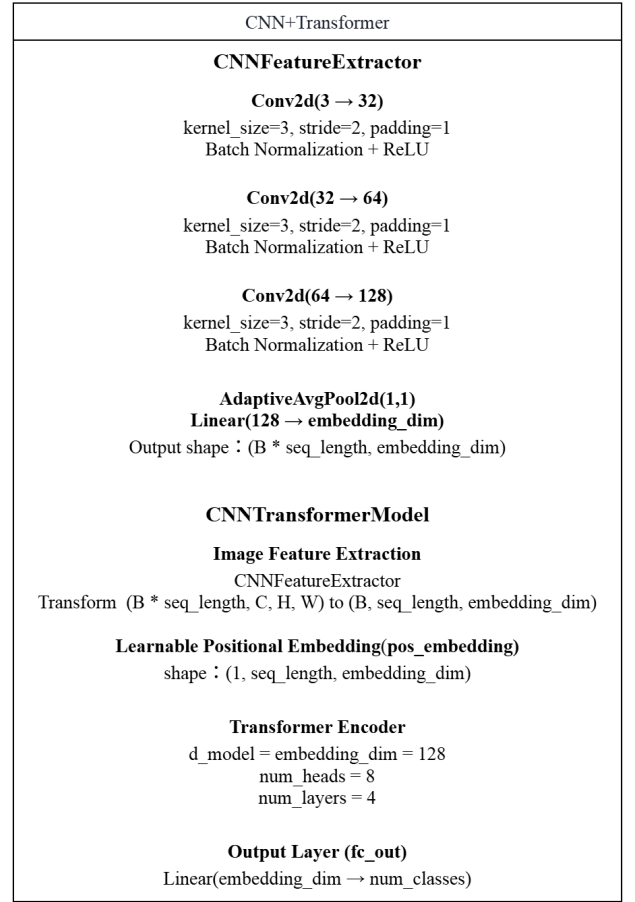


Fig. 6: System workflow diagram.

During the training phase, the model is trained for 5 epochs using CrossEntropyLoss as the loss function and optimized using the Adam optimizer with a learning rate of 0.001. This configuration ensures stable training and robust generalization capability even under limited data conditions.

12) *Model Post-processing*: After model prediction, the average predicted value for each class (no hit, red center, blue rim, silent segment) is calculated. A hit point is classified into a corresponding category if its predicted value exceeds the calculated average for that class. This approach dynamically

selects appropriate decision thresholds based on the model’s output distribution, effectively improving the accuracy and consistency of final annotations. Additionally, this method avoids detection biases caused by fixed thresholds, resulting in annotations that more closely align with actual rhythmic characteristics.

D. Evaluation Metrics

To objectively evaluate the structural and musical quality of the generated charts, two evaluation metrics are designed and adopted, addressing randomness differences, overall similarity, and temporal alignment:

- **DCRand (Direct Comparison against Random)**

This metric converts the model-generated charts into numerical representations of 0, 1, and 2, and then compares them point-by-point against randomly generated noise charts, calculating the proportion of matches at each timestep. DCRand primarily measures structural differences between the model output and purely random results. If the model predictions were random, the match rate would approach 0.50. Conversely, if the model successfully learns rhythmic structures, the output will significantly differ from randomness, with a match rate clearly deviating from 0.50. Therefore, DCRand values farther from 0.50 indicate a higher structural accuracy and closer alignment with human-designed charts.

- **OCHuman (Onset Comparison with Scale against Human)**

Since the chart design of *Taiko no Tatsujin* divides each measure into 16 equal segments for fixed hit-point positions, minor temporal deviations typically occur between musical rhythms and performance, causing slight misalignments between chart hits and actual rhythms. To address these minor timing discrepancies, the OCHuman metric introduces a ± 1 timestep tolerance window. This approach enhances the fairness and practicality of the evaluation, ensuring results more accurately reflect player perception and game experience.

IV. RESULTS

The model output is presented in a game-readable format using fixed-length numerical encoding as adopted by the .tja specification. Each line corresponds to a measure, with a fixed length of 16 segments, representing events such as center hits, rim hits, or silence. Specifically, “1” denotes a red note (center hit, Don), “2” represents a blue note (rim hit, Katsu), and “0” indicates no hit event. The encoded output is visualized using tools described in [23], reconstructing the chart as it would appear in-game, as shown in Fig. 7. In this visualization, red and blue circles denote Don and Katsu respectively, with each row representing a measure arranged from top to bottom, mimicking the scroll view in the game, as seen in Fig. 8.

This visual result enables multi-level analysis. From the overall layout, the generated chart exhibits natural variation in rhythmic density and clear segmentation, suggesting the model’s understanding of musical phrasing. Patterns such as consecutive hits (e.g., multiple Dons or Katsus), alternating strokes, and intentional silences demonstrate regularity and a human-like playing style. Certain measures show symmetrical distributions and heightened note density, corresponding to

musical climaxes or transitions, validating the model’s grasp of rhythmic structures.

This visualization not only provides intuitive insight into the rhythmic and structural consistency of the model output but also serves as a subjective structural evaluation beyond quantitative metrics like DCRand and OCHuman. The clear segmentation and variation in rhythm reflect common design logic in *Taiko no Tatsujin* charts, indicating the feasibility of the proposed model in generating human-like percussive patterns.

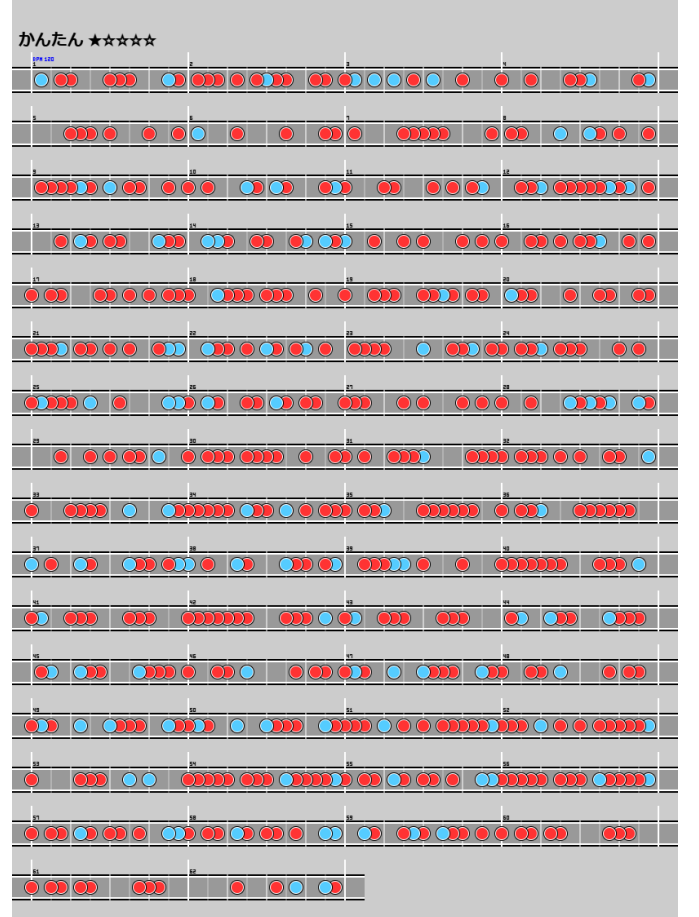


Fig. 7: Generated chart visualization example.

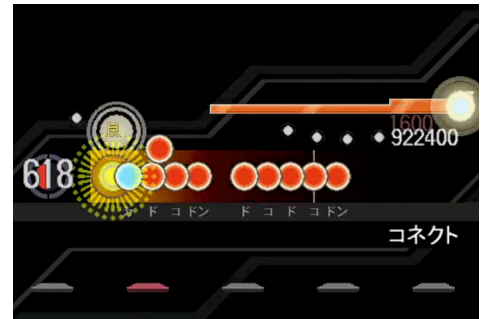


Fig. 8: Visualization of in-game scrolling interface.

V. DISCUSSION

A. Quantitative Results Comparison

In this study, a CNN-Transformer architecture was employed as the core model for automated chart generation

experiments conducted on a dataset consisting of 30 songs. The proposed model was benchmarked against the representative models, TaikoNation and Dance Dance Convolution (DDC), originally introduced in the TaikoNation study. Performance evaluation was carried out using two metrics: Direct Comparison against Random (DCRand), measuring structural similarity to random patterns, and Onset Comparison with Human (OCHuman), assessing the accuracy of predicted rhythmic onsets relative to human annotations.

For a fair comparison with prior works, we utilized the 10 benchmark test songs provided by the TaikoNation study as our evaluation dataset [24]. These songs were initially selected to evaluate a CNN-LSTM architecture within TaikoNation and are representative samples covering diverse rhythmic characteristics. However, the original TaikoNation dataset was derived from the osu! platform, whose chart annotations follow a timestamp-based format, where each drum hit corresponds to an absolute timestamp. In contrast, our study adopts the arcade version of *Taiko no Tatsujin*, which segments rhythmic events into sixteenth-note grids, dividing each measure uniformly into 16 subdivisions. To ensure compatibility and enable accurate comparison of predictions across these distinct formats, we developed a specialized conversion mechanism. This mechanism translates the osu! timestamp-based annotations into positions aligned with the sixteenth-note rhythmic grids, thus maintaining consistency with the training and inference format of our BU model, while preserving the original rhythmic structures and density characteristics.

As shown in Table I, the BU model achieved an average OCHuman score of 0.8426, surpassing both TaikoNation (0.7932) and DDC (0.8345). This indicates the effectiveness of the proposed rhythmic processing and data alignment strategy in improving the model’s rhythmic tolerance. Regarding the DCRand metric, the proposed BU model exhibited significant structural distinctiveness (0.4756), diverging substantially from the random baseline of 0.50. This suggests that the model successfully learned meaningful rhythmic patterns. Conversely, TaikoNation and DDC generated outputs that were closer to random noise, reflecting relatively weaker rhythmic structures. Overall, the BU model demonstrated superior rhythmic structural coherence, generating charts that are both structured and closely aligned with human charting logic.

B. Model Behavior Analysis and Limitations

The BU model demonstrated strong structural modeling capabilities, leveraging self-attention mechanisms to effectively capture long-range rhythmic dependencies. The model performed notably well in the OCHuman metric, maintaining high playability and structural coherence, even in densely populated or rhythmically complex sections. This underscores its proficiency in integrating global temporal features and accurately identifying rhythmic transitions and variations.

However, limitations were observed under conditions of limited data and sparse annotations, leading to issues such as attention dispersion and unstable learning within the Transformer architecture. Analysis of certain outputs revealed occasional rhythmic confusion or asymmetric patterns, particularly in overly dense or sparsely annotated regions. Moreover,

TABLE I: Average Performance of Proposed and Baseline Models

Metric	BU	TaikoNation [10]	DDC [8]
DCRand	0.4756	0.5041	0.4994
OCHuman	0.8426	0.7932	0.8345

the model’s lack of explicit music theory constraints (such as strong and weak beats or repeating sections) resulted in some generated charts appearing rhythmically consistent yet musically monotonous and lacking hierarchical depth.

Additionally, while spectrograms and labels were standardized through padding mechanisms, the manual specification of padding (e.g., blank images or special one-hot encodings) occasionally impeded the model’s ability to clearly differentiate between actual and padded data. This ambiguity contributed to prediction drift, especially evident in reduced accuracy towards the end of generated sequences.

C. Future Research Directions

Although the BU model has demonstrated stable structural recognition and acceptable rhythmic accuracy in small-to-medium datasets, there remains room for further expansion and optimization in future research:

- **Incorporation of Advanced Music Theory Features:** Integrate rhythm-level information (such as beat strength and phrase symmetry) or employ music grammar modeling (e.g., Hidden Markov Models, Context-Free Grammars) to enhance structural coherence.
- **Improvement of Model Architecture:** Introduce multi-scale attention mechanisms, rhythm-oriented positional encoding, or combine convolutional architectures with temporal pyramid structures to address attention dispersion issues, particularly in sparse data scenarios.
- **Subjective and Interactive Evaluation Mechanisms:** Design playability surveys, simulate player interaction behaviors, or collect player feedback to complement quantitative metrics, capturing real-world user experiences.

Overall, the CNN-Transformer architecture, combined with the proposed annotation and padding mechanisms, has shown promising potential in generating readable and rhythmically coherent charts for rhythm games.

The BU model’s strong rhythmic sensitivity, as evidenced by its superior performance on the OCHuman metric, suggests future studies could benefit from incorporating rhythm-aware encoding, pattern-aware loss functions, rhythm segment labeling mechanisms, or multi-scale attention models to enhance understanding of long-range rhythmic structures and improve model stability.

Furthermore, given the sensitivity of Transformer architectures to data volume and annotation density, it is recommended to utilize larger and more diverse standard chart datasets during training to mitigate attention drift and overfitting issues, thereby enhancing practical application performance.

VI. CONCLUSION

This study proposed and implemented an automated chart generation system for the rhythm game *Taiko no Tatsujin*.

Through systematic data preprocessing, spectrogram conversion, and annotation-padding strategies, the model effectively addressed challenges such as data sparsity, sequence length inconsistency, and alignment errors. The core architecture, a CNN combined with Transformer, was strengthened by the proposed segmentation and padding approach, which improved the model's capacity to capture temporal onsets and global rhythmic structures.

Experimental results confirmed that the BU model achieved outstanding performance on the OCHuman metric, reaching a maximum score of 0.9729, demonstrating its feasibility and potential in rhythm game applications. Nonetheless, the Transformer architecture remains susceptible to attention drift and rhythmic instability when facing sparse or limited training data.

Future research directions include the integration of rhythm-aware encoding, pattern-aware loss functions, and multi-scale attention mechanisms to improve the model's structural understanding and human-like drumming pattern generation. Additionally, expanding dataset scale, increasing annotation density, and incorporating subjective playability assessments and user interaction analysis are recommended to further enhance real-world applicability and system robustness.

References

- [1] J. S. Downie, “Music information retrieval,” *Annual Review of Information Science and Technology*, vol. 37, pp. 295–340, 2003. doi: 10.1002/aris.1440370108.
- [2] G. Masataka and H. Keiji, “Recent studies on music information processing,” *Acoustical Science and Technology*, vol. 25, no. 6, pp. 419–425, 2004. doi: 10.1250/ast.25.419. [Online]. Available: <https://doi.org/10.1250/ast.25.419>.
- [3] M. Schedl, E. Gómez, and J. Urbano, “Music information retrieval: Recent developments and applications,” *Foundations and Trends in Information Retrieval*, vol. 8, no. 2-3, pp. 127–261, 2014. doi: 10.1561/15000000042. [Online]. Available: <https://doi.org/10.1561/15000000042>.
- [4] Z. Lin, K. Xiao, and M. Riedl, “Generationmania: Learning to semantically choreograph,” in *Proceedings of a conference on AIIDE (2018)*, 2018.
- [5] N. (B. N. Amusement), 太鼓の達人 (*taiko no tatsujin*), 街機遊戯, 首次稼働日: 2001□02□21, 日本, 2001. [Online]. Available: <https://taiko.namco-ch.net/taiko/>.
- [6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [7] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [8] C. Donahue, Z. C. Lipton, and J. McAuley, *Dance dance convolution*, arXiv:1703.06891, 2017. [Online]. Available: <https://arxiv.org/abs/1703.06891>.
- [9] C. E. Achkar, “Music encoding and deep learning for music transcription and classification based on visually represented audio features,” Ph.D. dissertation, HAL Science Theses, 2023. [Online]. Available: <https://reurl.cc/xNOAve>.
- [10] E. Halina and M. R. Guzdial, *Taikonation: Patterning-focused chart generation for rhythm action games*, 2021. [Online]. Available: <https://arxiv.org/abs/2107.12506>.
- [11] T.-H. Chang, *Automatic generation of taiko games using deep learning methods*, 2022. [Online]. Available: <https://reurl.cc/rroolk>.
- [12] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” *arXiv preprint*, vol. arXiv:1508.07909, 2016. [Online]. Available: <https://doi.org/10.48550/arXiv.1508.07909>.
- [13] Y. Wang, “Application and analysis of deep learning on sleep quality prediction tasks,” in *2024 6th International Conference on Frontier Technologies of Information and Computer (ICFTIC)*, 2024. doi: 10.1109/ICFTIC64248.2024.10913081. [Online]. Available: <https://doi.org/10.1109/ICFTIC64248.2024.10913081>.
- [14] B. Hamzaoui, D. Bouchiha, and A. Bouziane, “A comprehensive survey on arabic text classification: Progress, challenges, and techniques,” *Brazilian Journal of Technology*, vol. 8, no. 1, 2025. doi: 10.38152/bjtv8n1-022. [Online]. Available: <https://doi.org/10.38152/bjtv8n1-022>.
- [15] K. Saarikka, V. Varsha, P. S. Harsha, and A. N. Sheikh, “Deep learning for automated image captioning: A cnn and transformer model analysis,” in *2025 3rd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*, 2025. doi: 10.1109/IDCIOT64235.2025.10914740. [Online]. Available: <https://doi.org/10.1109/IDCIOT64235.2025.10914740>.
- [16] S. Jaiswal, H. Pallthadka, and R. P. Chinchewadi, “Hybrid vision transformers and cnns for enhanced image captioning with beam search optimization,” *Technical Gazette*, 2024. [Online]. Available: <https://reurl.cc/j9ko9M>.
- [17] C. Welling, B. Longborough, and B. Winterbourne, *Semantic layer reconstruction in large language models using multi-level transformer feedback mechanisms*, 2024. [Online]. Available: <https://reurl.cc/YY9p4a>.
- [18] X. Song, Q. Niu, J. Liu, B. Peng, S. Zhang, and M. Liu, *Transformer: A survey and application*, 2024. [Online]. Available: <https://reurl.cc/1K2qvW>.
- [19] M. Zhang and D. Liu, “Cnn – lstm based multimodal models for music generation,” in *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, 2024. doi: 10.1109/ISPA63168.2024.00117.
- [20] Y. F. Wu, X. Liu, X. Gan, and W. Li, “Xbeat: A hybrid cnn-transformer model for beat and downbeat tracking,” in *National Conference on Sound and Music Computing*, 2023. [Online]. Available: <https://reurl.cc/lz9AyA>.
- [21] C. Wu, R. Wang, S. Lu, *et al.*, “Time-series data-driven pm. forecasting: From theoretical framework to empirical analysis,” *Atmosphere*, vol. 16, no. 3, p. 292, 2025. doi: 10.3390/atmos16030292. [Online]. Available: <https://doi.org/10.3390/atmos16030292>.
- [22] Yonokid, *Ese: Tjadatataba 譜面処理工具*, <https://ese.tjadatataba.se/ESE/ESE>, Accessed: 2025-07-24, 2025.
- [23] 太鼓さん次郎有志ユーザー, 太鼓さん次郎 (*tja*) 関連ツール, [https://wikiwiki.jp/taiko/□□□□□□\(TJA\)□□□□□□](https://wikiwiki.jp/taiko/□□□□□□(TJA)□□□□□□), Accessed: 2025-07-15, n.d.
- [24] D. Herbert *et al.*, *Beatmap listing: Osu!* <https://osu.ppy.sh/beatmapsets>, Accessed: 2025-07-24, 2007.